SCORE: A Framework for Synthesizing Complete, Owned, Responsive Environments

J.P. Kardolus, Babon Innovations B.V., Utrecht, The Netherlands

Working draft, 21 October 2026

Abstract

Games, films and robot training need 3D environments that are complete, editable and owned by their creators; world models and one-shot generators give frames no engine opens, or one fused scene without separate objects or clear rights. Studios already build such environments by a standard: scenes layered in OpenUSD, each scene sitting within its parent scene and each department's work on it a layer of its own that can be checked and changed apart from the rest. SCORE builds environments by this convention. The creator decides from the top down: he locks an environment concept (EC), its style and its physics, which holds the scenes and their layering, and then a concept for each scene, in order of parent scene and child scene. The framework makes every layer below with open vision foundation models, procedural tools and coding agents, each layer checked before the next is made. Every model allows commercial use of its output, so the creator stays the author. We describe the framework with its first environment as the worked example; its results are marked gaps until its remake has run. Code and paper are available at https://github.com/Babon-Innovations-b-v/score (MIT license).

Gap: Figure 1, one environment top down. The EC of environment 1 (its style and its physics), a top scene on its site, a parent scene and its child scenes, and one scene's layers side by side. It waits on the remake's batch.

1 Introduction

Interactive 3D environments are the raw material of games, film and embodied AI [1], and building one is expensive: every object is modelled, surfaced, placed, given collision, lit and given sound in one style, and the environment stays editable. Generative models promise to take over much of this, but in GDC's 2026 survey 52 percent of game workers thought generative AI was hurting the industry [2]; a tool for building environments has to leave the result in the creator's hands.

Studios already have a standard for this: OpenUSD [3], the scene description Pixar made for its films. Each artist works in a file of their own, a layer; the layers are stacked in a stated order, so departments work on one scene without editing each other's work, and scenes are assembled from smaller ones by reference. In this convention a scene sits within its parent scene and holds its child scenes, and each department's work on it (layout, set dressing, surfaces, light, effects, motion, sound) is a layer that can be checked and changed apart from the rest. SCORE builds environments by this convention, with the creator deciding and the framework doing the departments' work. The creator locks an environment concept (EC): its style and its physics. The EC holds the scenes and their layering. For each scene a concept is then locked, in order of parent scene and child scene, and from it the framework makes every layer below, each its own OpenUSD layer, checked before the next is made.

World models generate an environment as frames [4], [5] that no engine can open. 3D environment generators [6], [7] produce geometry, but mostly as one fused asset, from which separate objects are recovered after the fact [8], [9]. Both work in one shot, so a creator cannot find or change the step behind a fault. Procedural systems [10], [11] and coding agents that write scenes as programs [12], [13], [14], [15], [16], [17] keep the environment explicit, but rebuild a scene that already exists or draw their objects from a fixed library; their models often float or come apart [18], and such agents judge their own geometry unreliably [19]; on scenes rebuilt from video, the best model scores 0.91 on static scenes but 0.67 on dynamic ones [20], which is why SCORE checks motion as a layer of its own. Open vision foundation models [21] now read shapes [22], masks [23], depth [24] and parts [25] out of pictures, and a coding agent can connect them as tools; none of them provides one style across objects that different models made, which the field names as an open problem [26], [27].

SCORE's environments are meant to be **complete**: every object, surface, light and sound exists wherever the viewer looks. They are **owned**: every model in the framework allows commercial use of its output, so the creator stays the author, holding the full commercial rights. They are meant to be **responsive**: every object has real collision, mass and material. And they can be any kind of **environment**, from a house on a plot of ground to a system of worlds. To our knowledge, no other openly available environment generation work joins vision foundation models into engine-ready environments that are built from reference pictures, steered by a person's choices at each step, and made only with models that allow commercial use of their output.

We contribute **the SCORE framework**, which builds an environment by the studios' convention, the creator deciding top down and the framework making every layer below (Section 2); **measured gates** for every layer of a scene and a completion gate over them (Section 3); and **evidence from a first environment**, left as gaps until its remake has run (Section 4).

2 The framework

The creator works in the creator cockpit, one signed-in web page: there he decides the design, buys paid work and reviews what the framework reports. Below him, an environment is a table of stages read off its designs; environment 1's has 261 rows today, 201 of them the bodies of the 67 people its designs show. A **node** is one stage for one scene (`assembly@hangar`) or for the environment, and one runner makes, judges and accepts every node. From an approved design everything runs without a further yes.

2.1 Keys, gates and accepts

Keys. A node is made only when its key has no finish record. The key hashes the stage and scene, its code version, contracts, parameters and seed, the pinned model images, every file it reads and the accepted output of every node it reads. The code version counts the code that can shape the output: the stage's maker, the scripts it sends to a machine and every module they import; code that only decides where and how a job runs (renting, the pool of warm machines, the ledger, money guards, the cockpit) changes what a run costs, never what it makes, and is in no key. A fault fixed in one stage moves only its key and those after it. Outputs are kept by content hash, never overwritten.

Gates and accepts. Each gate answers pass, fail or unknown, with measures and a reason; missing evidence is unknown, and unknown blocks like fail. The runner accepts an output by itself when every gate passed and it is no worse than the one accepted before: no answer dropped, no measure

moved the wrong way by a step a viewer could see, no gate missing. Later nodes read only accepted outputs. The creator keeps four actions, buttons in the cockpit: reject an accepted output (it moves back to the one before, and every node that read it is stale), accept a worse one with a reason, waive one gate for one key, and override the runner for one paid run. A barrier keeps agents from them: a store that refuses the creator's records from an agent's process, and a hook.

Batches. Paid work is bought as a batch, with one estimate and one ETA; the creator's yes to that estimate is the only spend decision, and the batch stops at its high estimate plus 10 percent. A scene enters a batch only once its design stands whole on its lock, and Buy waits until the film script is locked. Inside a batch every paid job runs once, with no retry; a failed output becomes a batch gap that the completion gate lists, and spending more on it is a new batch. The creator reviews each scene in a 3D viewer on its accepted assembly.

2.2 From concept to scene

The environment concept. The creator first locks the EC, in a short interview with Gemini 3.1 Pro [28] in the cockpit: its style and its **physics**, with his reference pictures. The EC holds the scenes and their layering: the top scenes and the child scenes each may hold. Its physics is what everything stands on and in and how it moves, at the scale he sets: a plot of ground with a gravity and a sun path, one world, or a system of bodies that move one another, a real body taking its values from a cited table and an invented one given by its radius, mass, orbit and spin. A body near enough to matter moves in the physics simulation, a far source of light is a light, and what is too far to move is a backdrop. An environment plan places each top scene's site on that ground, shaped by that scene's own plan, and works out the sky each site sees.

A concept for each scene, top down. For each scene a concept is then locked, in order of parent scene and child scene. Each draw is one Nano Banana Pro [29] call that the creator's click pays for, on a concept review page where he marks areas and asks for a new draw, or first discusses the draw with Gemini 3.1 Pro, the thread going with his next draw. A child scene is drawn only once its parent scene is locked and planned, on a crop of the parent's concept, so it fits the parent it stands in.

Inventory and plan. The creator's lock is the approval. Gemini 3.1 Pro lists every object of the locked concept as a row of the scene inventory, with its kind, count, anchor and size in metres, and SAM 3 [23] boxes each row on the concept. A row counts when both find it, or when Gemini names one large object (at least 2 percent of the picture) that SAM 3 misses, boxed by Gemini's region; a row without metres is a gap. Nothing is made or placed that is not a row. The plan is made in two calls. Nano Banana Pro first draws the scene from straight above, from the locked concept, in the frame of the scene's own proportions; Gemini 3.1 Pro then writes the plan in metres from that drawing, the concept and the inventory, element by element: outline, walls, openings, levels, stairs, roofs, terrain heights, each item with its facing, people as the spots they stand on, and the child scenes' rooms. Free checks follow (every row placed as often as it counts, inside the outline, no two floor items in each other, each item placed and facing as in the concept, every door and standing spot reached on foot, the child scenes tiling their parent). What code can settle is fixed before the plan is shown, what remains goes back to Gemini 3.1 Pro as notes at most twice, and only what is left is a gap. A plan the creator drew himself wins over a written one, and his edits on the plan review page always win and redo what follows: he moves and resizes rooms, and a box he drops wholly inside a room is a thing in it, one dropped over another room makes the two one shared room, and one dropped anywhere else is a new child scene. A child scene he has locked keeps its room when its parent's plan is written again. Only a scene with a design gets the rows of a batch.

Shapes first. The route is one loop, the same at every scale. A parent scene gets its concept, its plan and a rough model, its concept put through picture to 3D whole, which gives the plan its proportions. Its plan is then split into the shapes of its child scenes, and once those shapes are fixed every child scene is made in parallel, each repeating the loop (the creator's order of 19 October 2026). The runner's rows `rough-model` and `shell-split` (gated on `tiled`, `reached` and `bounded`) are built and have not run.

The scene record. The scene record holds what no made piece covers (shells, stairs, ground and water, backdrop, lamps, the neighbours a scene shows), written from the plan, inventory and made ground, and gated on `game-free`, `plan-matched` (the shell within 5 cm of the plan) and `openings` (every doorway onto a floor, no view into nothing).

The canonical scene. Every scene is one OpenUSD stage [3]; each object, one per real-world object, carries its kind, transform, collision and each part's library surface. Its layers are read off its design alone: each inventory row's kind, found free from the reading's noun, says what its objects do (emit, move, lie loose, glow, grow, act, sound), a scene gets an effects, motion, physics, crops or people layer only for rows that do so, and light and sound always. Each layer is made from the rows by their kind's rule, never from a record written for one scene, so a new environment needs no hand-written list. The layers stack in a fixed order: a slot for the creator's edit layer on top, then people, sound, light, motion, effects, crops, machines, settle, ground and the base, laid from the locked plan. Each scene is assembled once from every layer's accepted output; its `layers` gate answers unknown while a layer its design shows has none. Walls, floors, stairs and ground collide and every model has a collider (`collision`); colliders are static, mass and interaction not yet written. **A parent scene holds its child scenes**: its scene tree references each child's own tree at the room its plan gives it, so opening a top scene opens all below it, each still its own layer stack (`child scenes`). Neighbours are referenced the same way; a shown scene no stage makes is a gap. The edit layer is still a slot: a test on a two-box scene shows an edit there surviving a regenerated base, but the assembly does not yet carry the creator's edits from one assembly to the next.

Models, generation first. Every object goes the prop route first: Nano Banana Pro draws its close-up from its crop of the concept (a picture the shape check rejects is drawn again, at most three times, on the creator's word of 24 October 2026), Pixal3D [22] builds it, the surface stage paints it, and a likeness gate compares its outline from eight sides with its close-up. A close-up whose every take fails the shape check is a batch gap, so Pixal3D is never paid for it. Only an object whose checks fail is built by a code builder a coding agent writes (the creator's choice, 12 October 2026). A big object the viewer walks up to gets a body built in code under the generated details its entry names (the creator, 18 October 2026).

Gap: Figures 2 and 3. (2) One scene's design top down: the EC, the parent scene's concept and plan, a child scene drawn on its parent's crop, its inventory and plan. (3) Objects of one scene from close-up to final model, generated and code-built. They wait on the remake's designs and batch.

3 Stages and their gates

3.1 Surfaces: the finish route

Everything a scene shows comes from the creator's library, score_lib, or is a written exception (the creator's rule of 18 October 2026). Models give shape only: a picture-made model keeps its geometry and its finish is replaced. Every part takes one surface from a shared library of rule-based

surfaces, written as ProcFunc functions [30] on Infinigen's shaders [10], coloured from the scene's palette and worn only where wear has a cause. The library starts empty for the remake: every surface and model of the earlier runs was retired on 21 October 2026, so the batch makes them fresh. Each scene's palette is coined from its locked concept and its objects' close-ups, one Gemini 3.1 Pro batch answer gives its surfaces as settings of the recipes, and an entry is reused only when made under today's route.

The finish route gives each model its finishes (the creator's choice of 20 October 2026). Free renders cover every face of the raw model, each view split into colour regions; one Gemini 3.1 Pro answer a model lists its finishes, each matched to a library surface, and labels each region, and the labels vote across views into one finish a face, a face no label reaches or whose views disagree being a gap. A face no view can see takes the finish of the nearest seen one, an island of a finish under 1 percent of the model joins the finish round it, and a model whose paint is still patchy, over 3 percent of it in such islands, is a gap before its bake. Each part is painted from its library surface and baked, and the bake is checked against the close-up: a finish's surface must lie within ΔE 32 of its colour there, a bake fails when it darkens a finish the close-up shows lighter or turns all surfaces into one grey, and a foot over 3 cm off the floor fails. A free library check traces every material, model, light, sound and character of a scene's accepted assembly to a library entry or a written exception.

3.2 Resting

Loose objects are dropped onto the made ground in a physics simulation; one that would turn more than 15 degrees or drift more than 30 cm is a resting fault of the layout and keeps its laid pose. The settle stage lays where each object comes to rest as a layer (held objects snapped onto their holders, a stack onto one support, an object that would rest worse kept as laid), gated on `rested` and `resting` (no fault that appears with the layer and not without it). A physics layer writes the scene's physics in OpenUSD, and a trial on Bullet drops the loose rows' objects where they stand, under the gravity of the body the scene stands on (`reacted`, `at rest`, `energy`, `hinge`).

3.3 Light

The sky is worked out from the EC's physics, never drawn: the light layer writes what the scene's top site sees overhead at its scene time, each body at its true direction, size and phase (`sky bodies`), and a light under the horizon lights nothing. The lamps are the inventory's glowing rows, and their strengths are measured. One paid render per scene gives every lamp its own Cycles light group on the scene's views; because light adds up, the look at any strengths is mixed from these passes without rendering again, and a free sweep chooses the scale that brings the scene's brightness within 1.5 times its concept's, halving any lamp that turns what it lights white.

3.4 Motion, effects and sound

Motion. Every object of a moving row moves by its kind's rule (a door swings and shuts, a crane slews, a fan spins) at the middle of its kind's speed range cited from real machines, gated on `clear` (no moving thing within 3 cm of a still thing or another moving thing, read every quarter second), `speeds`, `loops`, `grounded`, `covered` and `designed`. **Effects:** an emitting row's objects carry their kind's emitters, measured against real footage, gated on `attached`, `footage`, `light`, `outside` (no open-sky particle inside a room) and `designed`. **Sound:** a scene's sources come from its design (a bed from how the scene is heard, its sounding rows, its people's voices and its moving things), and each gets a library file or one MOSS-SoundEffect [31] take of its brief, gated on `sources`, `bed`,

`events`, `voices`, `level`, `likeness` (CLAP [32] score at least 0.30; a take under it is a gap) and `no freesound`.

3.5 People

People are models like any other. The looks come from the person rows of the locked designs, each person from its own SAM 3 box on the concept and a crowd as one far look; the shared loop draws its close-up (one person, front on, in the rig's A-pose), Pixal3D builds it and the finish route paints it. The rigged body takes each part's library surface from that paint, its face and hair included, and the library surfaces are baked onto the body itself as onto any model and checked the same way; no picture's finish reaches it. SOMA-X's body, read from the close-up by SAM 3D Body [33], is fitted under it as its rig, and Kimodo's clips [34], from sentences, move it. The **shell gate** passes a look when 99 percent of its soft points lie within 5 cm of the body, its worst point recorded; each clip gets its own verdict, and casts use only passing clips. **Casts:** each person stands on a person spot of its scene's plan, gated on `counted`, `standing`, `clear` and `made`.

3.6 Completion

A scene is complete when its gate passes on its final assembly: every row made or dropped with a reason, nothing unmade, no placeholder or resting fault, the creator's picks shown and every layer and shown scene present. A fault the creator decides to ship with is an accepted gap, written with who decided, when, why and its measured faults; the completion gate lists every gap by name and is never loosened, and a scene whose accepted gaps are its only fail is complete apart from accepted gaps. A hook stops an agent claiming a scene is done while its gate fails: agents judging their own scenes agree with a measured geometry score only 45.8 percent of the time [19].

3.7 The film

The film is a design step too: the creator locks a film script in the cockpit (by default the whole environment, body by body), and a scene's films are made from locked scripts only, once it is complete or complete apart from accepted gaps.

Gap: Figures 4 to 7. (4) A scene's models painted from the library beside their close-ups; (5) a scene swept to its concept's light; (6) motion and effects in place; (7) its people and their cast. They wait on the remake's assemblies.

3.8 Prices and ledgers

Every paid stage is priced from a unit cost measured on the ledgers, times the units its forecast rule counts, times the bill's share over the ledger; a stage with no measured run has no price, and its estimate is shown apart as a guess. Today 15 of the table's 29 paid kinds of row have a measured unit price, four of them from two benchmark batches of 21 October 2026 that the creator approved, one node a kind. Machines and Google calls write ledger lines with their node's clearance, and the month's spend is the higher of ledgers and bill, which caught misses: EUR 1,554 billed against EUR 1,387 in the cloud ledger on 12 October 2026. A paid node runs only when every gate on its inputs passed, the month's books are known and its batch's stop has room for its forecast.

4 Evidence

Environment 1 is the worked example. Its first run was not accepted by the creator and its store was lost on 19 October 2026, so this draft shows none of its outputs; its remake from new designs has not run.

Gap: Table 1 and the film. Per scene of the remake: inventory rows, models generated and coded, layers, gaps before and after the batch, time, euros and the creator's verdict; and a walk through environment 1. They wait on the remake's batch and the creator's review.

Gap: Table 2, people. Usable clips per look by the shell and clip gates on the remake's people. It waits on the people line's batch.

Agent tools. With them an agent used about half the output and cost on two of three scenes; on the third it left more faults; one run each.

5 Limitations, future work and conclusion

One creator judges, and he has not yet accepted a run. Closed models (Nano Banana Pro, Gemini 3.1 Pro) draw the concepts and close-ups and read and plan every scene; they may change or go. The design step, the layers made from row kinds, the fresh library, the finish route, the shapes-first loop, the scene tree and the code bodies have not yet carried a scene through a batch; the edit layer is a slot, colliders are static and objects are not interactive, and no game engine opens the result yet. Next: the remake's batch, an engine adapter with physics and interaction, an edit layer the assembly carries, three more environments, open models, LEGO-Bench [19]. In short, SCORE builds an environment the way studios do, scene by scene and layer by layer, so a fault is fixed in its own layer; an accepted environment will show if it holds.

References

[1] Y. Yang *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[2] D. Argüello, "36 percent of game workers use generative AI, but half think it's bad for the industry." Game Developer, Feb. 2026.

[3] Pixar Animation Studios and Alliance for OpenUSD, "OpenUSD: Universal scene description." 2026. Available: https://openusd.org/

[4] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, *et al.*, "Genie: Generative interactive environments," *arXiv preprint arXiv:2402.15391*, 2024.

[5] J. Parker-Holder and S. Fruchter, "Genie 3: A new frontier for world models." Google DeepMind blog, Aug. 2025. Available: https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

[6] D. Wang, H. Jung, T. Monnier, K. Sohn, *et al.*, "WorldGen: From text to traversable and interactive 3D worlds," *arXiv preprint arXiv:2511.16825*, 2025.

[7] Tencent Hunyuan, "HY-World 2.0." GitHub repository, 2026. Available: https://github.com/Tencent-Hunyuan/HY-World-2.0

[8] J. Hu, J. Guo, J. Cen, C. Yang, S. Li, and W. Shen, "WorldAct: Activating monolithic 3D worlds into interactive-ready object-centric scenes," *arXiv preprint arXiv:2605.15843*, 2026.

[9] M. Niu, J. He, R. Yu, L. Fu, *et al.*, "WorldSculpt: Generating compositional worlds from grounded videos," *arXiv preprint arXiv:2609.05416*, 2026.

[10] A. Raistrick *et al.*, "Infinite photorealistic worlds using procedural generation," *arXiv preprint arXiv:2306.09310*, 2023.

[11] A. Raistrick *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[12] Z. Hu, A. Iscen, A. Jain, T. Kipf, *et al.*, "SceneCraft: An LLM agent for synthesizing 3D scene as Blender code," in *International conference on machine learning (ICML)*, 2024.

[13] Z. Li, Y. Liao, and B. Zhu, "Recursive code world models: Building complex worlds through recursive scene programs," *arXiv preprint arXiv:2609.11499*, 2026.

[14] C. Guo, J. Li, Y. Li, and Z. Huang, "WorldClaw: Agentic 3D open-world generation at scale," *arXiv preprint arXiv:2608.05248*, 2026.

[15] L. Yin, W. Cheng, Z. Qin, T. Huang, *et al.*, "AutoUE: Automated generation of 3D games in Unreal Engine via multi-agent systems," *arXiv preprint arXiv:2603.07106*, 2026.

[16] G. He, R. Luo, W.-C. Ma, and H. Averbuch-Elor, "Thinking in Blender: Staged executable inverse graphics with vision-language models," *arXiv preprint arXiv:2606.02580*, 2026.

[17] H. Wang *et al.*, "Code as worlds: Agentic discovery of executable world representations for physical reasoning," *arXiv preprint arXiv:2608.27549*, 2026.

[18] Y. Gao *et al.*, "3DCodeBench: Benchmarking agentic procedural 3D modeling via code," *arXiv preprint arXiv:2606.01057*, 2026.

[19] X. Li *et al.*, "LEGO-Anything: Coding agents for 3D scene reconstruction," *arXiv preprint arXiv:2609.36380*, 2026.

[20] R. Shen *et al.*, "4DCodeBench: Benchmarking agents on inverse graphics of dynamic scenes," *arXiv preprint arXiv:2610.03715*, 2026.

[21] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[22] D.-Y. Li *et al.*, "Pixal3D: Pixel-aligned 3D generation from images," *arXiv preprint arXiv:2605.10922*, 2026.

[23] N. Carion *et al.*, "SAM 3: Segment Anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[24] R. Wang *et al.*, "MoGe-2: Accurate monocular geometry with metric scale and sharp details," *arXiv preprint arXiv:2507.02546*, 2025.

[25] Y. Lin *et al.*, "PartCrafter: Structured 3D mesh generation via compositional latent diffusion transformers," in *Advances in neural information processing systems (NeurIPS)*, 2025.

[26] J. Wu, Z. Lou, J. Liu, D. Du, C. Guo, and S. Guo, "From visual synthesis to interactive worlds: Toward production-ready 3D asset generation," *arXiv preprint arXiv:2604.23629*, 2026.

[27] Z. Yang *et al.*, "FlowScene: Style-consistent indoor scene generation with multimodal graph rectified flow," *arXiv preprint arXiv:2603.19598*, 2026.

[28] Google, "Gemini 3.1 Pro (gemini-3.1-pro-preview) through the gemini API." 2026. Available: https://ai.google.dev/

[29] Google, "Gemini 3 Pro Image ('Nano Banana Pro') through the Gemini API." 2026. Available: https://ai.google.dev/

[30] A. Raistrick *et al.*, "ProcFunc: Function-oriented abstractions for procedural 3D generation in Python," *arXiv preprint arXiv:2604.26943*, 2026.

[31] OpenMOSS-Team, "MOSS-SoundEffect v2.0." Hugging Face, 2026. Available: https://huggingface.co/OpenMOSS-Team/MOSS-SoundEffect-v2.0

[32] Y. Wu, K. Chen, T. Zhang, *et al.*, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," *arXiv preprint arXiv:2211.06687*, 2022.

[33] X. Yang, D. Kukreja, D. Pinkus, others, and K. Kitani, "SAM 3D Body: Robust full-body human mesh recovery," *arXiv preprint arXiv:2602.15989*, 2026.

[34] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, *et al.*, "Kimodo: Scaling controllable human motion generation," *arXiv preprint arXiv:2603.15546*, 2026.